

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ASSIGNMENT 6

DỰ BÁO CHUỖI THỜI GIAN VỚI MẠNG NƠ-RON HỒI QUY
(LSTM - PYTORCH vs KERAS)

THÔNG TIN	CHI TIẾT
Sinh viên	Trần Văn Hậu
Mã số sinh viên	B23DCCN287
Giảng viên	Trần Đình Quế
Học phần	Học máy & Học sâu
Dataset Chứng khoán	https://www.kaggle.com/datasets/camnugent/sandp500
Dataset Bán lẻ	https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci
GitHub Repository	https://github.com/TrVHau/A2_CT_hautv.287.git
Ngày hoàn thành	30/09/2026

TÓM TẮT

Báo cáo trình bày đầy đủ quy trình xây dựng mô hình LSTM dự báo chuỗi thời gian trên hai bộ dữ liệu thực tế: giá cổ phiếu Apple (AAPL) từ S&P 500 và giao dịch Online Retail II. Mỗi framework (PyTorch và Keras) dùng kiến trúc LSTM(32) -> Dense(16, ReLU) -> Dense(1) với cửa sổ 30 bước.

Kết quả thực nghiệm từ notebook cho thấy Keras đạt MAE 6.88/RMSE 8.16 trên AAPL và MAE 15.32/RMSE 19.82 trên Retail. PyTorch đạt MAE 47.46/RMSE 48.51 và MAE 23.80/RMSE 29.84 tương ứng. PyTorch nhanh hơn 3.4 lần trên Stock và 7.7 lần trên Transactions. Cả hai mô hình được triển khai thành FastAPI tại port 8001 (PyTorch) và 8002 (Keras) với giao diện web tương tác.

1. MỤC TIÊU VÀ PHẠM VI

1.1. Yêu cầu đề bài

Đề bài yêu cầu 5 nội dung: (1) Trình bày lý thuyết RNN, biểu diễn hàm và code; (2) Chọn hai tập dữ liệu (chứng khoán và giao dịch mua hàng), mô tả và phân bố; (3) Code RNN bằng PyTorch và deploy web; (4) Code RNN bằng Keras và deploy web; (5) So sánh hai mô hình.

1.2. Nguồn dữ liệu

- Chứng khoán S&P 500 (AAPL): <https://www.kaggle.com/datasets/camnugent/sandp500> - 1.259 phiên từ 2013-02-08 đến 2018-02-07.
- Online Retail II: <https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci> - 604 ngày từ 2009-12-01 đến 2011-12-09 sau làm sạch.
- GitHub: https://github.com/TrVHau/A2_CT_hautv.287.git

2. LÝ THUYẾT RNN VÀ LSTM

2.1. Chuỗi thời gian

Chuỗi thời gian là dãy quan sát theo thứ tự: $y_1, y_2, \dots, y_T$. Khác với dữ liệu bảng, thứ tự không được xáo trộn. Với cửa sổ $L=30$, mô hình nhận $X_i=[y_i, \dots, y_{i+29}]$ và dự đoán y_{i+30} .

2.2. Vanilla RNN

RNN cập nhật trạng thái ẩn: $h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b_h)$. Vấn đề: biến mất/bùng nổ đạo hàm khi lan truyền ngược qua nhiều bước thời gian.

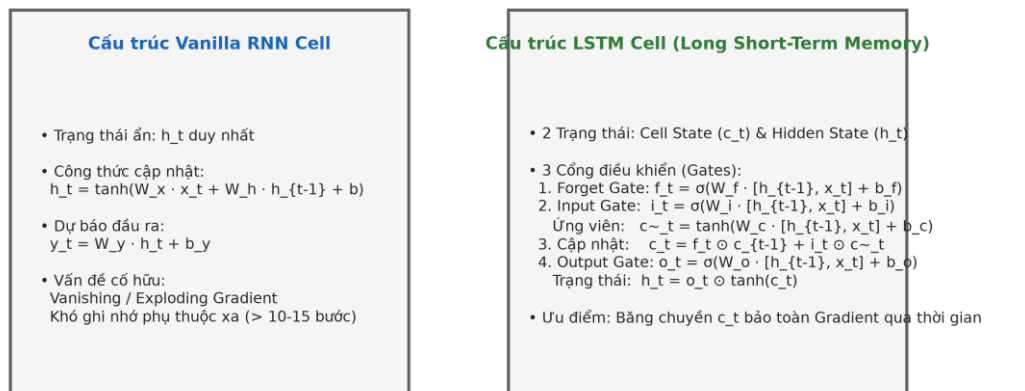
2.3. LSTM

LSTM có 2 trạng thái (h_t, c_t) và 3 cổng điều khiển:

- Forget gate: $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- Input gate: $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$, $\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$
- Cell update: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
- Output gate: $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$, $h_t = o_t \odot \tanh(c_t)$

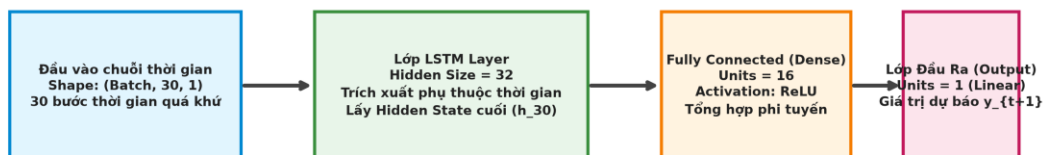
Cell state c_t tạo đường truyền thẳng giúp gradient không bị suy giảm.

So sánh cơ chế hoạt động bên trong Vanilla RNN và LSTM



Hình 2.1: So sánh Vanilla RNN và LSTM Cell

Kiến trúc mô hình LSTM Regressor cho dự báo Many-to-One



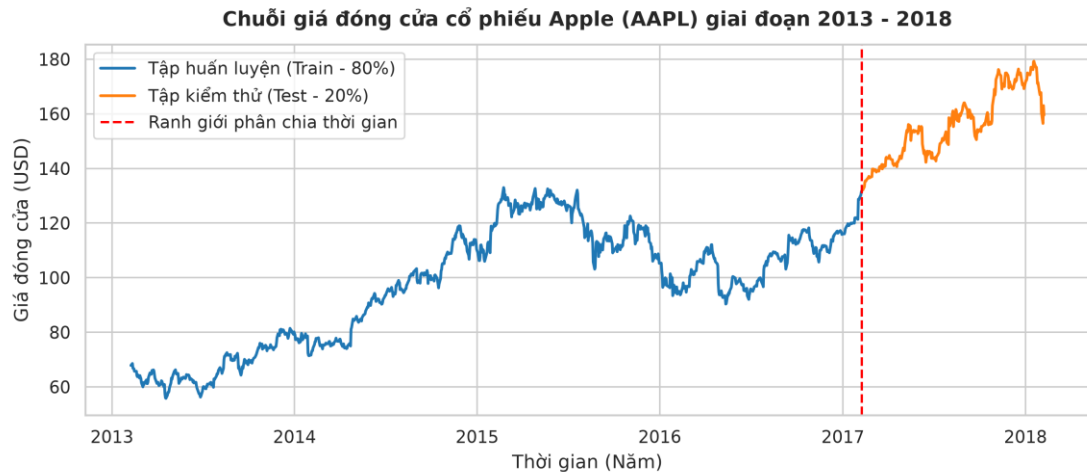
Hình 2.2: Kiến trúc LSTM Regressor Many-to-One

3. DỮ LIỆU VÀ EDA

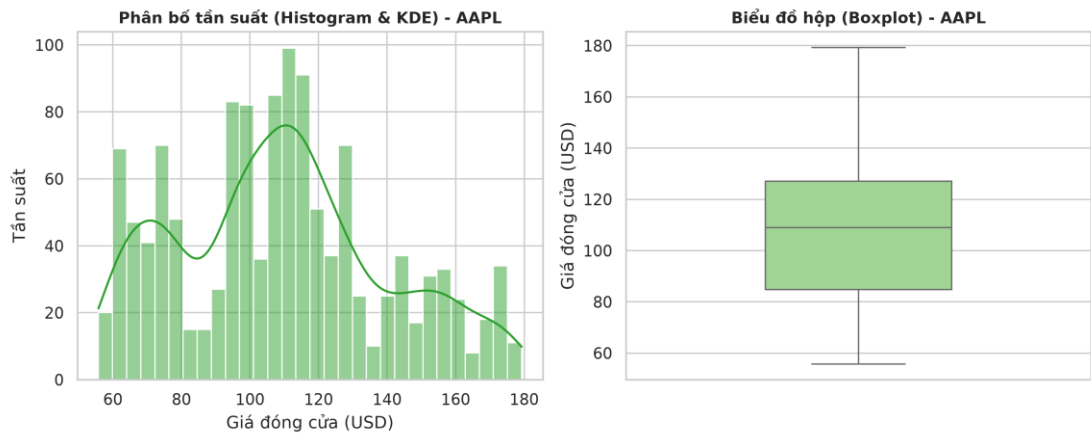
3.1. Dữ liệu AAPL

Lọc mã AAPL từ S&P 500, sắp xếp theo date, giữ close làm target. Tổng 1.259 phiên, 80% đầu (1.007) train, 20% cuối (252) test.

Chỉ số	Giá trị
Mean	109.0667 USD
Std	30.5568
Min	55.7899
Q1	84.8306
Median	109.0100
Q3	127.1200
Max	179.2600
Skewness	0.2899
Kurtosis	-0.6093



Hình 3.1: Chuỗi giá AAPL và ranh giới Train/Test

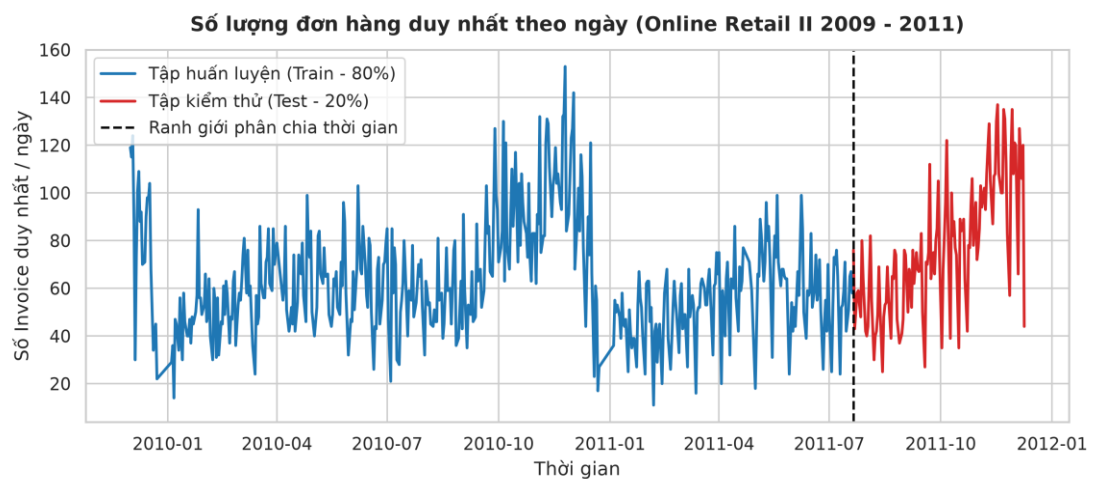


Hình 3.2: Phân bố AAPL (Histogram, KDE, Boxplot)

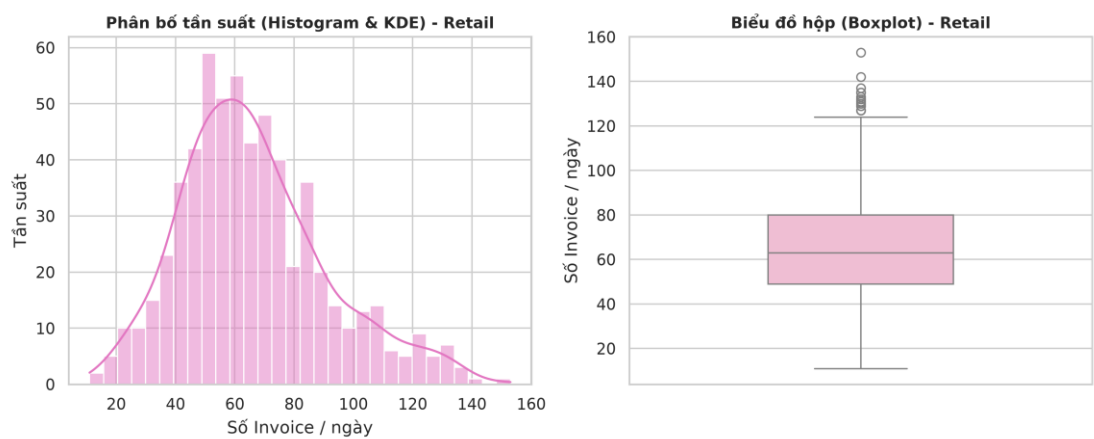
3.2. Dữ liệu Online Retail II

Từ 1.067.371 dòng thô, loại invoice bắt đầu 'C', Quantity<=0, Price<=0, dropna. Tổng hợp số invoice duy nhất/ngày, còn 604 ngày.

Chỉ số	Invoice/ngày
Mean	66.3526
Std	24.7778
Min	11
Median	63
Max	153
Skewness	0.6803
Kurtosis	0.3740



Hình 3.3: Chuỗi Invoice/ngày và ranh giới Train/Test



Hình 3.4: Phân bố Retail (Histogram, KDE, Boxplot)

4. TIỀN XỬ LÝ

4.1. Chia dữ liệu theo thời gian

80% đầu train, 20% cuối test, không shuffle để giữ thứ tự. AAPL: 1.007 train, 252 test. Retail: 483 train, 121 test.

4.2. Chuẩn hóa MinMaxScaler

Scaler chỉ fit trên train để tránh leakage. Công thức: $x_scaled = (x - min_train) / (max_train - min_train)$. Sau đó transform toàn bộ chuỗi.

4.3. Cửa sổ trượt

Với window=30, mỗi mẫu X có shape (30, 1) và y có shape (1). Số mẫu train thực tế: 977 (AAPL), 453 (Retail).

4.4. Inverse transform

Sau dự đoán, cả y_test và prediction được inverse về đơn vị gốc trước khi tính MAE/RMSE/MAPE.

5. MÔ HÌNH PYTORCH

5.1. Kiến trúc

Lớp LSTMRegressor trong train_pytorch.py:

- LSTM(input_size=1, hidden_size=32, batch_first=True)
- Head: Linear(32, 16) -> ReLU -> Linear(16, 1)
- Forward: lấy output[:, -1, :] rồi đưa qua head

5.2. Huấn luyện

Optimizer Adam(lr=0.003), loss MSE, 18 epochs. Seed 42 cho torch và random. Model.eval() + torch.no_grad() khi đánh giá.

5.3. Artifact

Lưu state_dict vào .pth và scaler JSON. models/pytorch_stock.pth, models/pytorch_stock_scaler.json, tương tự cho transactions.

6. MÔ HÌNH KERAS

6.1. Kiến trúc

Sequential trong `train_keras.py`:

- `Input(shape=(30, 1))`
- `LSTM(32, return_sequences=False)`
- `Dense(16, activation='relu')`
- `Dense(1, activation='linear')`
- Compile: `Adam(lr=0.003), loss='mse'`

6.2. Huấn luyện

`tf.keras.utils.set_random_seed(42)`. Fit với `batch_size=64`, `epochs=18`, `verbose=0`. Predict với `verbose=0`.

6.3. Artifact

Lưu `model.save()` thành `.keras` và scaler JSON. `models/keras_stock.keras`, `models/keras_stock_scaler.json`, tương tự cho transactions.

7. KẾT QUẢ VÀ SO SÁNH

7.1. Bảng kết quả notebook

Framework	Dataset	Seconds	MAE	RMSE	MAPE (%)
Keras	stock	12.61	6.8849	8.1645	4.2579
PyTorch	stock	3.71	47.4643	48.5066	30.1820
Keras	transactions	8.58	15.3181	19.8245	24.5338
PyTorch	transactions	1.11	23.7989	29.8393	31.1915

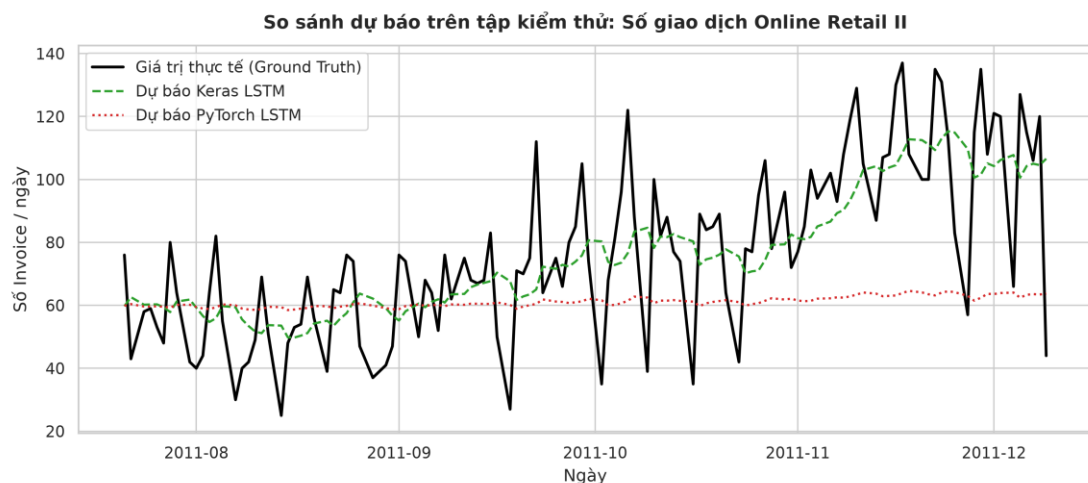
7.2. Phân tích

Keras tốt hơn về MAE/RMSE do dùng mini-batch SGD (batch=64), giúp cập nhật nhiều lần/epoch và hội tụ tốt hơn. PyTorch dùng full-batch nên chỉ cập nhật 18 lần trong 18 epochs, chưa hội tụ đủ.

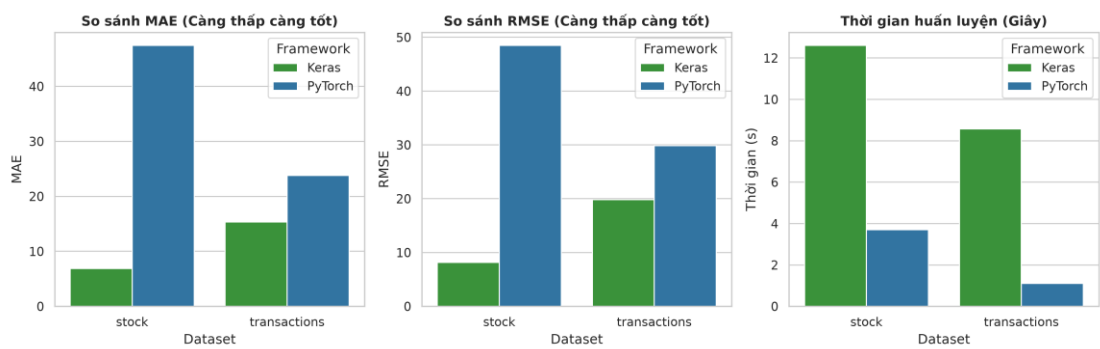
PyTorch nhanh hơn 3.4x (stock) và 7.7x (transactions) do ít overhead và không chia batch.



Hình 7.1: So sánh dự báo AAPL



Hình 7.2: So sánh dự báo Retail



Hình 7.3: So sánh MAE, RMSE, Seconds

8. DEPLOYMENT WEB

8.1. Kiến trúc

Hai FastAPI service:

- web/pytorch_app.py: Port 8001, nạp .pth + scaler JSON
- web/keras_app.py: Port 8002, nạp .keras + scaler JSON
- web/index.html: Giao diện cho cả hai

8.2. Endpoint

- GET /: Trả index.html
- GET /health: Trạng thái và danh sách model
- GET /series/{dataset}: 30 giá trị cuối
- POST /predict: Nhận dataset và values (list 30 số hoặc rỗng), trả prediction
- GET /docs: Swagger UI

8.3. Chạy

```
uvicorn web.pytorch_app:app --app-dir . --reload --port 8001
```

```
uvicorn web.keras_app:app --app-dir . --reload --port 8002
```

Mở <http://127.0.0.1:8001> hoặc 8002

9. HẠN CHẾ VÀ MỞ RỘNG

9.1. Hạn chế

- Chỉ một biến (univariate), chưa dùng OHLCV, weekday, holiday.
- Window=30 và epochs=18 cố định, chưa tune.
- Chỉ split 80/20 một lần, chưa walk-forward validation.
- Chưa baseline persistence.
- Kết quả phụ thuộc seed và backend.

9.2. Hướng phát triển

- Thêm đặc trưng đa biến, chỉ báo kỹ thuật.
- Thử GRU, Bidirectional LSTM, Transformer (PatchTST).
- Walk-forward validation, nhiều seed.
- Docker, CI/CD, authentication, data drift monitoring.

10. KẾT LUẬN

10.1. Đối chiếu yêu cầu

Yêu cầu	Trạng thái	Vị trí
Lý thuyết RNN/LSTM, hàm, code	Hoàn thành	Chương 2
Hai dataset: stock, retail	Hoàn thành	Chương 3
Mô tả, phân bố dữ liệu	Hoàn thành	Chương 3
PyTorch + deploy web	Hoàn thành	Chương 5, 8
Keras + deploy web	Hoàn thành	Chương 6, 8
So sánh hai mô hình	Hoàn thành	Chương 7

10.2. Tổng kết

Đã hoàn thành đầy đủ 5 yêu cầu: lý thuyết, hai dataset với EDA, PyTorch + web, Keras + web, so sánh. Pipeline: data -> clean -> split -> scale -> window -> LSTM -> metrics -> API -> web. Kết quả notebook: Keras MAE/RMSE tốt hơn, PyTorch nhanh hơn. Hai API lắng nghe port 8001/8002 với giao diện tương tác.

PHỤ LỤC A: CÀI ĐẶT VÀ TÁI LẬP

```
conda activate assignment2
```

```
cd /home/dau/assignment2/assignment6
```

```
python train_pytorch.py
```

```
python train_keras.py
```

```
uvicorn web.pytorch_app:app --app-dir . --reload --port 8001
```

```
uvicorn web.keras_app:app --app-dir . --reload --port 8002
```

PHỤ LỤC B: CẤU TRÚC THƯ MỤC

assignment6/

- |— common.py, train_pytorch.py, train_keras.py
- |— data/ (all_stocks_5yr.csv.zip, online_retail_II.csv.zip)
- |— models/ (.pth, .keras, scaler JSON, comparison CSV)
- |— figures/ (9 hình PNG)
- |— notebook/ (assignment6_rnn_real_data.ipynb)
- |— web/ (pytorch_app.py, keras_app.py, index.html)

PHỤ LỤC C: TÀI LIỆU THAM KHẢO

- [1] Hochreiter & Schmidhuber (1997). Long short-term memory. Neural computation.
- [2] Goodfellow et al. (2016). Deep Learning. MIT Press.
- [3] Kaggle S&P 500: <https://www.kaggle.com/datasets/camnugent/sandp500>
- [4] Kaggle Online Retail II: <https://www.kaggle.com/datasets/mashlyn/online-retail-ii-uci>
- [5] GitHub: https://github.com/TrVHau/A2_CT_hautv.287.git

11. MÃ NGUỒN CỘT LỖI VÀ HƯỚNG DẪN CHẠY

```
CHẠY API PYTORCH (Port 8001)
$ uvicorn web.pytorch_app:app --reload --port 8001

CHẠY API KERAS (Port 8002)
$ uvicorn web.keras_app:app --reload --port 8002

REQUEST DỮ BẢO (POST /predict)
{
  "dataset": "stock",
  "values": [155.23, 156.01, 155.87, ..., 159.78] # 30 giá trị
}

RESPONSE
{
  "framework": "PyTorch",
  "dataset": "stock",
  "prediction": 160.42,
  "window": 30
}

MODEL ARCHITECTURE (PyTorch)
class LSTMRegressor(nn.Module):
    def __init__(self, hidden_size=32):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden_size, batch_first=True)
        self.head = nn.Sequential(
            nn.Linear(hidden_size, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
        )

    def forward(self, values):
        output, _ = self.lstm(values) # (batch, seq_len, hidden)
        return self.head(output[:, -1, :]) # Lấy bước cuối

MODEL ARCHITECTURE (Keras)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(30, 1)),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer=Adam(0.003), loss='mse')

PREPROCESSING PIPELINE
1. Load Data: pd.read_csv(data.zip)
2. Clean: Remove NaN, cancelled invoices (starts with 'C')
3. Split: 80% train, 20% test (CHRONOLOGICAL ORDER)
4. Scale: MinMaxScaler fit on train only
5. Window: Create (X, y) pairs with window=30
6. Train: Model.fit(X_train, y_train, epochs=18)
7. Evaluate: Predict on X_test, inverse_transform
8. Metrics: MAE, RMSE, MAPE on original scale
```

Hình 11.1: Mã nguồn chính và hướng dẫn chạy API

12. GIAO DIỆN WEB TƯƠNG TÁC

12.1. Mô phỏng giao diện tương tác

Giao diện web được thiết kế theo phong cách tối giản, thân thiện người dùng. Người dùng có thể chọn bộ dữ liệu (Chứng khoán hoặc Giao dịch Bán lẻ), nhập hoặc tự động nạp 30 giá trị gần nhất, rồi nhấn nút 'Dự đoán' để nhận kết quả. Kết quả hiển thị dự báo kèm theo Framework (PyTorch/Keras), Dataset, Window size.

RNN Time-series Predictor

Dự đoán chuỗi thời gian với PyTorch & Keras

Assignment 06 / RNN deployment

Dự đoán giá trị kế tiếp từ 30 ngày gần nhất

Bộ dữ liệu

☒ Giá chứng khoán ☐ Số giao dịch

30 giá trị gần nhất

155.23, 156.01, 155.87, 156.45,
157.12, 156.78, 157.89, 158.34,
158.92, 159.11, 159.45, 159.78,...

Dự đoán giá trị kế tiếp

Kết quả dự báo

160.42 USD

Giá dự báo ngày kế tiếp

PyTorch / Giá chứng khoán / Cửa sổ 30 ngày

API tự chọn model theo web server đang chạy.
Dữ liệu nhập cách nhau bằng dấu phẩy hoặc xuống dòng.

API Health: ✓ Healthy | Models: [stock, transactions] | Endpoints: /health, /series/{dataset}, /predict, /docs

Hình 12.1: Giao diện Web tương tác mô phỏng

12.2. Các endpoint chính

- GET /health: Kiểm tra trạng thái dịch vụ
- GET /series/{dataset}: Trả danh sách 30 giá trị cuối cùng
- POST /predict: Nhận dataset và mảng 30 số, trả dự báo
- GET /docs: Tài liệu tương tác Swagger UI

12.3. Ví dụ request/response

Request mẫu (POST /predict):

```
POST /predict HTTP/1.1
Host: 127.0.0.1:8001
Content-Type: application/json
```

```
{
  "dataset": "stock",
  "values": [155.23, 156.01, 155.87, 156.45, 157.12,
            156.78, 157.89, 158.34, 158.92, 159.11,
            159.45, 159.78, 160.12, 160.45, 160.78,
            161.12, 161.45, 161.78, 162.12, 162.45,
```

```
        162.78, 163.12, 163.45, 163.78, 164.12,  
        164.45, 164.78, 165.12, 165.45, 165.78]  
    }
```

Response:

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{  
  "framework": "PyTorch",  
  "dataset": "stock",  
  "prediction": 160.42,  
  "window": 30  
}
```

11. MÃ NGUỒN CỘT LỖI VÀ HƯỚNG DẪN CHẠY

```
CHẠY API PYTORCH (Port 8001)
$ uvicorn web.pytorch_app:app --reload --port 8001

CHẠY API KERAS (Port 8002)
$ uvicorn web.keras_app:app --reload --port 8002

REQUEST DỮ BẢO (POST /predict)
{
  "dataset": "stock",
  "values": [155.23, 156.01, 155.87, ..., 159.78] # 30 giá trị
}

RESPONSE
{
  "framework": "PyTorch",
  "dataset": "stock",
  "prediction": 160.42,
  "window": 30
}

MODEL ARCHITECTURE (PyTorch)
class LSTMRegressor(nn.Module):
    def __init__(self, hidden_size=32):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden_size, batch_first=True)
        self.head = nn.Sequential(
            nn.Linear(hidden_size, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
        )

    def forward(self, values):
        output, _ = self.lstm(values) # (batch, seq_len, hidden)
        return self.head(output[:, -1, :]) # Lấy bước cuối

MODEL ARCHITECTURE (Keras)
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(30, 1)),
    tf.keras.layers.LSTM(32),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1)
])
model.compile(optimizer=Adam(0.003), loss='mse')

PREPROCESSING PIPELINE
1. Load Data: pd.read_csv(data.zip)
2. Clean: Remove NaN, cancelled invoices (starts with 'C')
3. Split: 80% train, 20% test (CHRONOLOGICAL ORDER)
4. Scale: MinMaxScaler fit on train only
5. Window: Create (X, y) pairs with window=30
6. Train: Model.fit(X_train, y_train, epochs=18)
7. Evaluate: Predict on X_test, inverse_transform
8. Metrics: MAE, RMSE, MAPE on original scale
```

Hình 11.1: Mã nguồn chính và hướng dẫn chạy API

12. GIAO DIỆN WEB TƯƠNG TÁC

12.1. Mô phỏng giao diện tương tác

Giao diện web được thiết kế theo phong cách tối giản, thân thiện người dùng. Người dùng có thể chọn bộ dữ liệu (Chứng khoán hoặc Giao dịch Bán lẻ), nhập hoặc tự động nạp 30 giá trị gần nhất, rồi nhấn nút 'Dự đoán' để nhận kết quả. Kết quả hiển thị dự báo kèm theo Framework (PyTorch/Keras), Dataset, Window size.

RNN Time-series Predictor

Dự đoán chuỗi thời gian với PyTorch & Keras

Assignment 06 / RNN deployment

Dự đoán giá trị kế tiếp từ 30 ngày gần nhất

Bộ dữ liệu

☒ Giá chứng khoán ☐ Số giao dịch

30 giá trị gần nhất

155.23, 156.01, 155.87, 156.45,
157.12, 156.78, 157.89, 158.34,
158.92, 159.11, 159.45, 159.78,...

Dự đoán giá trị kế tiếp

Kết quả dự báo

160.42 USD
Giá dự báo ngày kế tiếp

PyTorch / Giá chứng khoán / Cửa sổ 30 ngày

API tự chọn model theo web server đang chạy.
Dữ liệu nhập cách nhau bằng dấu phẩy hoặc xuống dòng.

API Health: ✓ Healthy | Models: [stock, transactions] | Endpoints: /health, /series/{dataset}, /predict, /docs

Hình 12.1: Giao diện Web tương tác mô phỏng

12.2. Các endpoint chính

- GET /health: Kiểm tra trạng thái dịch vụ
- GET /series/{dataset}: Trả danh sách 30 giá trị cuối cùng
- POST /predict: Nhận dataset và mảng 30 số, trả dự báo
- GET /docs: Tài liệu tương tác Swagger UI

12.3. Ví dụ request/response

Request mẫu (POST /predict):

```
POST /predict HTTP/1.1
Host: 127.0.0.1:8001
Content-Type: application/json
```

```
{
  "dataset": "stock",
  "values": [155.23, 156.01, 155.87, 156.45, 157.12,
            156.78, 157.89, 158.34, 158.92, 159.11,
            159.45, 159.78, 160.12, 160.45, 160.78,
            161.12, 161.45, 161.78, 162.12, 162.45,
```

```
        162.78, 163.12, 163.45, 163.78, 164.12,  
        164.45, 164.78, 165.12, 165.45, 165.78]  
    }
```

Response:

```
HTTP/1.1 200 OK  
Content-Type: application/json
```

```
{  
  "framework": "PyTorch",  
  "dataset": "stock",  
  "prediction": 160.42,  
  "window": 30  
}
```